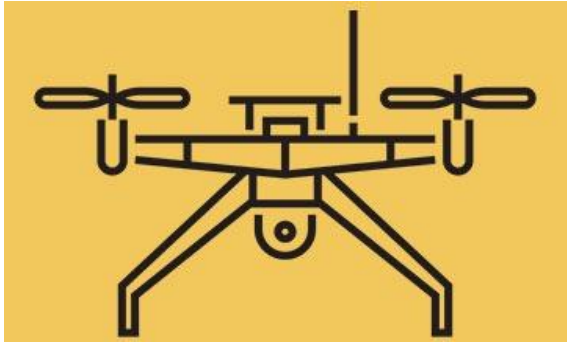
The Unscented Kalman Filter

Jerome Jouffroy, PhD
jerome@sdu.dk

Recap on the KF: Elements of probability theory



Consider the random variable X (a continuous variable)

Then the probability of the event X between a and b is given by

$$\Pr[a \leq X \leq b] = \int_a^b \mathrm{pr}(x)dx$$

probability density function

Property: $$\int_{-\infty}^{\infty} \mathrm{pr}(x)dx = 1$$

Expectation: $$\mathrm{E}[x] = \int_{-\infty}^{\infty} x\,\mathrm{pr}(x)dx = \mu$$

Variance: $$\mathrm{Var}[x] = \mathrm{E}[(x - \mathrm{E}[x])^2] = \sigma^2$$

Multivariate normal distribution:

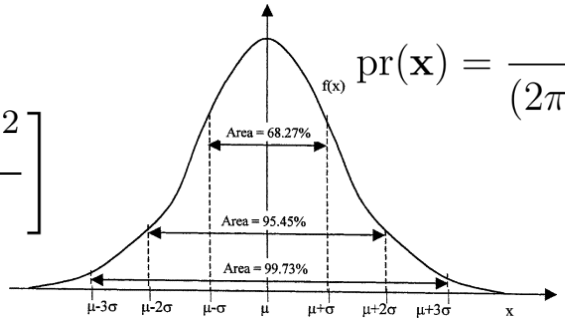
$$\mathbf{X} = [X_1, X_2, \cdots, X_n]^T$$

$$\mathrm{pr}(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{n}{2}}|\boldsymbol{\Sigma}|^{\frac{1}{2}}} \exp\left[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right]$$

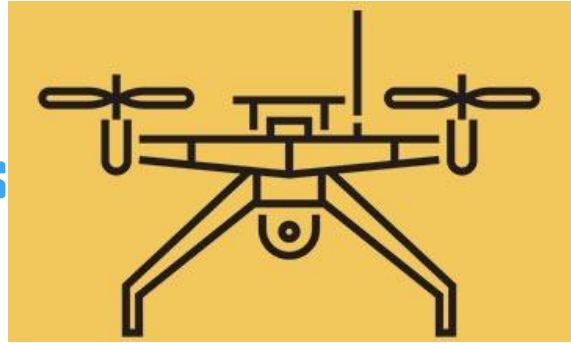
covariance matrix

Normal probability density function:

$$\mathrm{pr}(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right]$$



Recap on the KF: Dynamical system and assumptions



For the discrete linear Kalman Filter, we consider systems of the form

$$\mathbf{x}(k+1) = \mathbf{A}(k)\mathbf{x}(k) + \mathbf{w}(k)$$

state vector is not measured and needs to be estimated, dim(x)=n

possibly time-varying matrix

process noise

with the measurement equation

$$\mathbf{y}(k) = \mathbf{C}(k)\mathbf{x}(k) + \mathbf{v}(k)$$

measurement (as obtained by sensors), dim(y)=m

measurement noise

Assumptions:

$\mathbf{w}(k)$ Gaussian white noise with $\mathrm{E}[\mathbf{w}(k)] = 0$

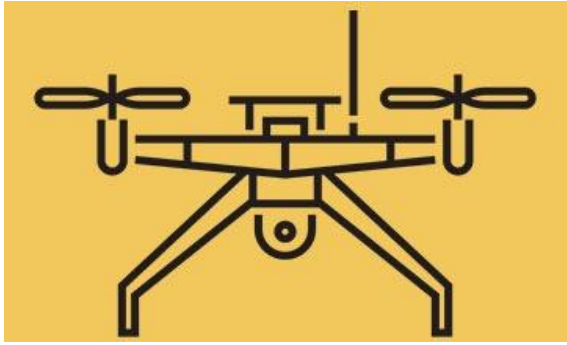
and $\mathrm{E}[\mathbf{w}(k)\mathbf{w}^T(i)] = \begin{cases} \mathbf{Q}(k) & \text{if } i = k \\ 0 & \text{if } i \neq k \end{cases}$

$\mathbf{v}(k)$ Gaussian white noise with $\mathrm{E}[\mathbf{v}(k)] = 0$

and $\mathrm{E}[\mathbf{v}(k)\mathbf{v}^T(i)] = \begin{cases} \mathbf{R}(k) & \text{if } i = k \\ 0 & \text{if } i \neq k \end{cases}$

$\mathbf{w}(k)$ and $\mathbf{v}(k)$ are uncorrelated: $\mathrm{E}[\mathbf{v}(k)\mathbf{w}^T(i)] = 0 \;\; \forall k, i$

Recap on the KF: the big picture



$$\hat{\mathbf{x}}_-(0)$$
$$\mathbf{P}_-(0)$$

Kalman gain

$$\mathbf{K} = \mathbf{P}_-(k)\mathbf{C}^T\left[\mathbf{C}\mathbf{P}_-(k)\mathbf{C}^T + \mathbf{R}\right]^{-1}$$

Correction/Update

Estimate correction/update

$$\hat{\mathbf{x}}(k) = \hat{\mathbf{x}}_-(k) + \mathbf{K}\left[\mathbf{y}(k) - \mathbf{C}\hat{\mathbf{x}}_-(k)\right]$$

Covariance correction/update

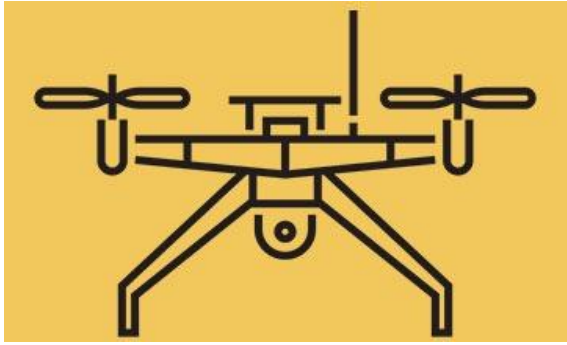
$$\mathbf{P}(k) = \left[\mathbf{I} - \mathbf{K}\mathbf{C}\right]\mathbf{P}_-(k)$$

Prediction

Prediction step

$$\begin{cases} \hat{\mathbf{x}}_-(k+1) = \mathbf{A}\hat{\mathbf{x}}(k) \\ \mathbf{P}_-(k+1) = \mathbf{A}\mathbf{P}(k)\mathbf{A}^T + \mathbf{Q} \end{cases}$$

Recap on the EKF



For the nonlinear system

$$\begin{cases} \mathbf{x}(k) = \mathbf{f}\left(\mathbf{x}(k-1)\right) + \mathbf{w}(k-1) \\ \mathbf{y}(k) = \mathbf{h}\left(\mathbf{x}(k)\right) + \mathbf{v}(k) \end{cases}$$

Prediction step

$$\begin{cases} \hat{\mathbf{x}}_-(k) = \mathbf{f}\left(\hat{\mathbf{x}}(k-1)\right) \\ \mathbf{P}_-(k) = \mathbf{A}\mathbf{P}(k-1)\mathbf{A}^T + \mathbf{Q} \end{cases}$$

with $\mathbf{A} = \left[\left.\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right|_{\mathbf{x}=\hat{\mathbf{x}}}\right]$

Kalman gain

$$\mathbf{K} = \mathbf{P}_-(k)\mathbf{C}^T\left[\mathbf{C}\mathbf{P}_-(k)\mathbf{C}^T + \mathbf{R}\right]^{-1}$$

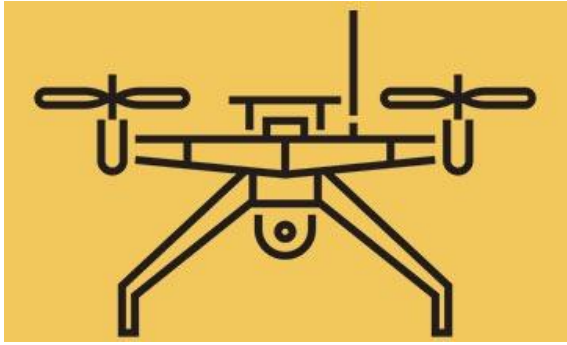
Correction/update step

$$\begin{cases} \hat{\mathbf{x}}(k) = \hat{\mathbf{x}}_-(k) + \mathbf{K}\left[\mathbf{y}(k) - \mathbf{h}\left(\hat{\mathbf{x}}_-(k)\right)\right] \\ \mathbf{P}(k) = \left[\mathbf{I} - \mathbf{K}\mathbf{C}\right]\mathbf{P}_-(k) \end{cases}$$

with $\mathbf{C} = \left[\left.\frac{\partial \mathbf{h}}{\partial \mathbf{x}}\right|_{\mathbf{x}=\hat{\mathbf{x}}}\right]$

<u>Problem</u>: making a linear approximation (ie using Jacobians A and C) can lead to a too crude approximation

A nonlinear static example



Consider a sonar sensor giving both range and bearing (r, θ) of a target situated at (0,1) in the cartesian plane

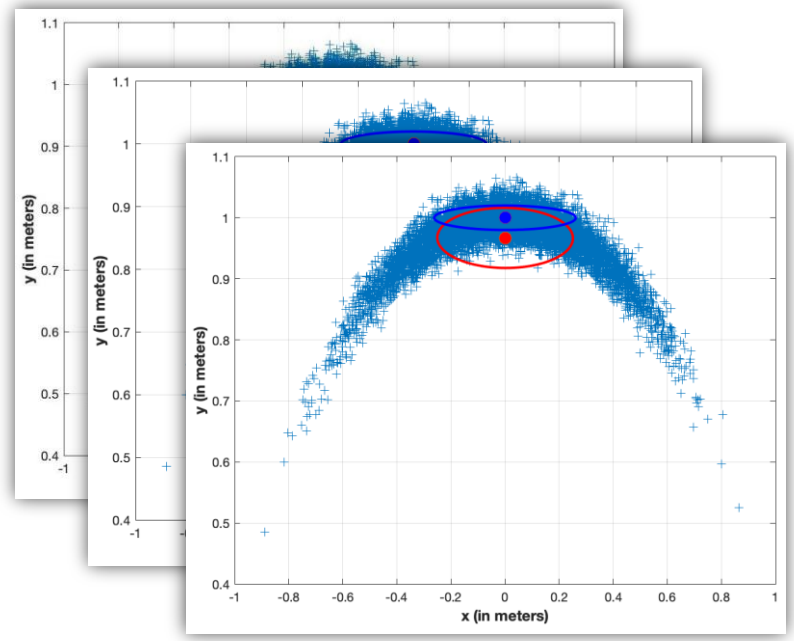
The measurements given by the sensor have the properties

$$\boldsymbol{\mu} = \begin{bmatrix} 1 \\ \pi/2 \end{bmatrix} = \begin{bmatrix} \bar{r} \\ \bar{\theta} \end{bmatrix} \quad \text{and} \quad \boldsymbol{\Sigma} = \begin{bmatrix} (0.02\text{m})^2 & 0 \\ 0 & (15\text{deg})^2 \end{bmatrix}$$

Transforming these measurements back into cartesian coordinates is given by

$$\mathbf{y} = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} r\cos\theta \\ r\sin\theta \end{bmatrix} = \mathbf{f}(\mathbf{x})$$

Having N=10000 gives this graph



A typical EKF-like approximation would give

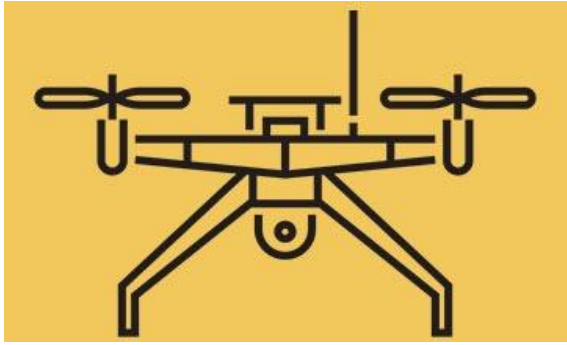
$$\begin{bmatrix} \hat{\bar{x}} \\ \hat{\bar{y}} \end{bmatrix} = \begin{bmatrix} \bar{r}\cos\bar{\theta} \\ \bar{r}\sin\bar{\theta} \end{bmatrix} \quad \text{and} \quad \hat{\boldsymbol{\Sigma}}_{\text{cart}} = \hat{\boldsymbol{\Sigma}}_{\mathbf{y}} = \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^T$$

$$\text{with} \quad \mathbf{A} = \begin{bmatrix} \cos\bar{\theta} & -\bar{r}\sin\bar{\theta} \\ \sin\bar{\theta} & \bar{r}\cos\bar{\theta} \end{bmatrix}$$

while in reality, we have

$$\bar{\mathbf{y}} = \boldsymbol{\mu}_{\mathbf{y}} = \mathrm{E}[\mathbf{y}(k)] \quad \text{and} \quad \boldsymbol{\Sigma}_{\text{cart}} = \boldsymbol{\Sigma}_{\mathbf{y}} = \mathrm{E}[(\mathbf{y}(k) - \bar{\mathbf{y}})(\mathbf{y}(k) - \bar{\mathbf{y}})^T]$$ which is very different!

The Unscented Transform (1/2)



<u>Main idea</u>: instead of using a linearisation to propagate a random variable through a nonlinear function, <u>take a few well-chosen sampling points to represent a Gaussian distribution</u>$(\boldsymbol{\mu}, \Sigma)$

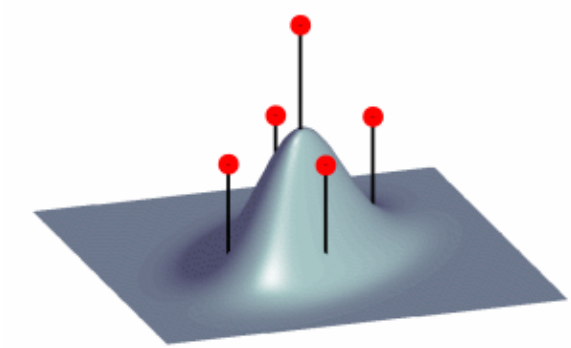
These sigma points $\boldsymbol{\mathcal{X}}^i$, together with their weighting factors w^i, should satisfy the following constraints

$$\boldsymbol{\mu} = \sum_i w^i \boldsymbol{\mathcal{X}}^i$$

$$\Sigma = \sum_i w^i (\boldsymbol{\mathcal{X}}^i - \boldsymbol{\mu})(\boldsymbol{\mathcal{X}}^i - \boldsymbol{\mu})^T$$

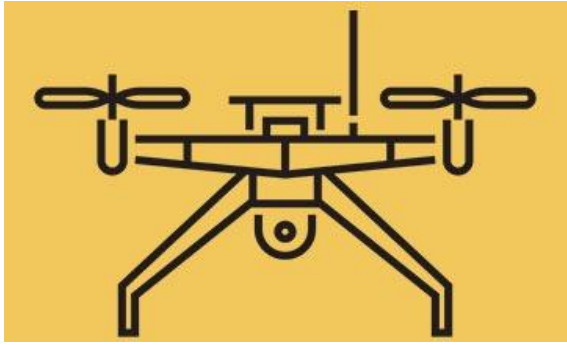
$$\sum_i w^i = 1$$

ie they represent a Gaussian distribution



There exists several ways to define sigma points and their corresponding weights...

The Unscented Transform (2/2)



Most used set of sigma points/weights:

For a distribution of dimension n, define 2n+1 sigma points:

$$\boldsymbol{\mathcal{X}}^0 = \boldsymbol{\mu}_{\mathbf{x}}$$

$$\boldsymbol{\mathcal{X}}^i = \boldsymbol{\mu}_{\mathbf{x}} + \left[\sqrt{(n+\kappa)\boldsymbol{\Sigma}_{\mathbf{x}}}\right]_i \quad \text{for i=1..n}$$

$$\boldsymbol{\mathcal{X}}^i = \boldsymbol{\mu}_{\mathbf{x}} - \left[\sqrt{(n+\kappa)\boldsymbol{\Sigma}_{\mathbf{x}}}\right]_{i-n} \quad \text{for i=n+1..2n+1}$$

scaling parameter

with the weights

$$w^0 = \frac{\kappa}{n+\kappa} \quad \text{and} \quad w^i = \frac{1}{2(n+\kappa)} \quad \text{for i=1..2n+1}$$

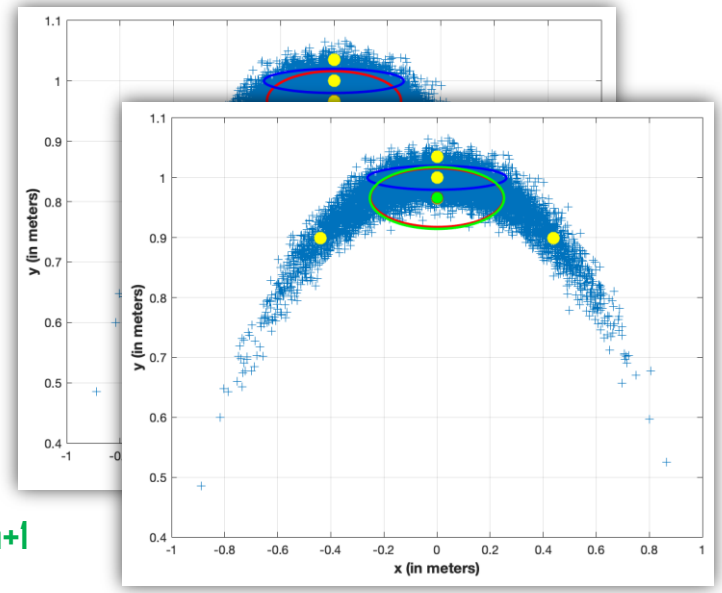
Take n=2:

$$\sum_1^5 w^i = \frac{\kappa}{2+\kappa} + \frac{4}{2(2+\kappa)} = \frac{\kappa+2}{2+\kappa} = 1$$

Transform each of these sigma points with

$$\boldsymbol{\mathcal{Y}}^i = \mathbf{f}(\boldsymbol{\mathcal{X}}^i)$$

For our sonar example:

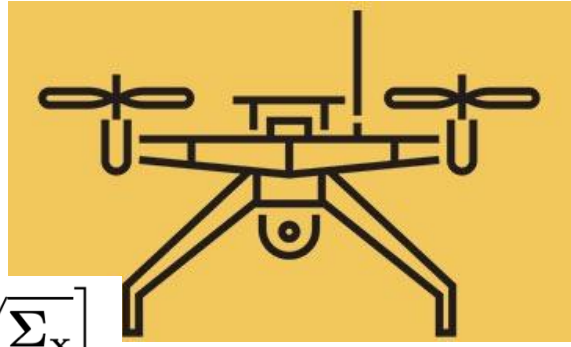


and compute

$$\boldsymbol{\mu}_{\mathbf{y}} = \sum_{i=0}^{2n} w^i \boldsymbol{\mathcal{Y}}^i$$

$$\boldsymbol{\Sigma}_{\mathbf{y}} = \sum_{i=0}^{2n} w^i (\boldsymbol{\mathcal{Y}}^i - \boldsymbol{\mu}_{\mathbf{y}})(\boldsymbol{\mathcal{Y}}^i - \boldsymbol{\mu}_{\mathbf{y}})^T$$

This gives a much better approximation!



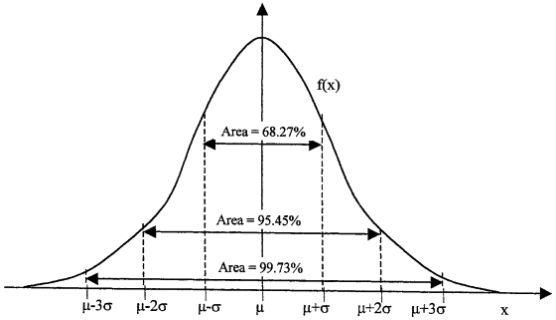
The matrix square root

Let us come back to

$$\boldsymbol{\mathcal{X}}^i = \boldsymbol{\mu}_{\mathbf{x}} + \left[\sqrt{(n+\kappa)\boldsymbol{\Sigma}_{\mathbf{x}}}\right]_i = \boldsymbol{\mu}_{\mathbf{x}} + \sqrt{(n+\kappa)}\left[\sqrt{\boldsymbol{\Sigma}_{\mathbf{x}}}\right]_i$$

What is $[\mathbf{A}]_i$ ➡ the i-th column vector of matrix A

What is $\sqrt{\boldsymbol{\Sigma}_{\mathbf{x}}}$? ➡ the square root of the covariance matrix



When n=1: $\sqrt{\boldsymbol{\Sigma}_{\mathbf{x}}} = \sigma_x$ standard deviation of Gaussian process

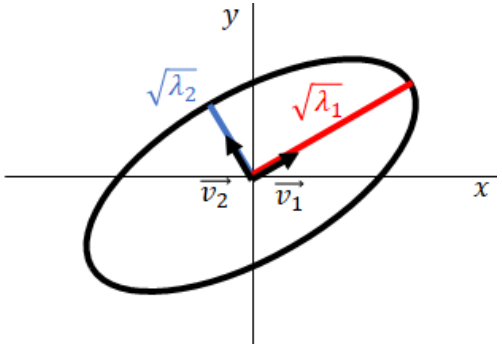
When n>1: The matrix square root L of $\boldsymbol{\Sigma}_{\mathbf{x}}$ is the matrix that satisfies $\boldsymbol{\Sigma}_{\mathbf{x}} = \mathbf{L}\mathbf{L}$

(in Matlab: command 'sqrtm')

One can also define slightly different sigma points which are proportional to eigenvectors: $\left[\mathbf{D}^{\frac{1}{2}}\right]_{i,i}[\mathbf{V}]_i = \sqrt{\lambda_i}\mathbf{v}_i = \sigma_i\mathbf{v}_i$

V matrix of eigenvectors

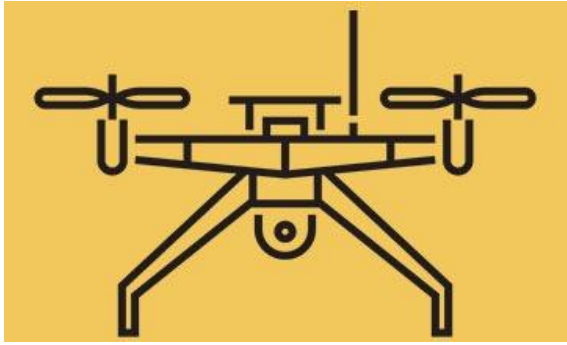
D matrix of eigenvalues



The above can lead to numerical errors...

Alternative: Cholesky factorization

➡ find L such that $\boldsymbol{\Sigma}_{\mathbf{x}} = \mathbf{L}^T\mathbf{L}$ (in Matlab: command 'chol')



UKF: the prediction step (1/2)

For the nonlinear system

$$\begin{cases} \mathbf{x}(k) = \mathbf{f}\left(\mathbf{x}(k-1)\right) + \mathbf{w}(k-1) \\ \mathbf{y}(k) = \mathbf{h}\left(\mathbf{x}(k)\right) + \mathbf{v}(k) \end{cases}$$

with

$$\mathbf{Q}(k) = \mathrm{E}[\mathbf{w}(k)\mathbf{w}^T(k)]$$
$$\mathbf{R}(k) = \mathrm{E}[\mathbf{v}(k)\mathbf{v}^T(k)]$$

Prediction/propagation: we need to find

$$\boxed{\begin{array}{l} \hat{\mathbf{x}}_-(k) \text{ from } \hat{\mathbf{x}}(k-1) \\ \mathbf{P}_-(k) \text{ from } \mathbf{P}(k-1) \end{array}}$$

Calculate sigma points:

$$\boxed{\begin{aligned} \boldsymbol{\mathcal{X}}^0(k-1) &= \hat{\mathbf{x}}(k-1) \\ \boldsymbol{\mathcal{X}}^i(k-1) &= \hat{\mathbf{x}}(k-1) \pm \sqrt{(n+\kappa)}\left[\sqrt{\mathbf{P}(k-1)}\right]_i \end{aligned}}$$

Propagate sigma points through the state equation:

$$\boxed{\boldsymbol{\mathcal{X}}_-^i(k) = \mathbf{f}(\boldsymbol{\mathcal{X}}^i(k-1))}$$

Calculate prior estimate as the weighted mean of the propagated sigma points:

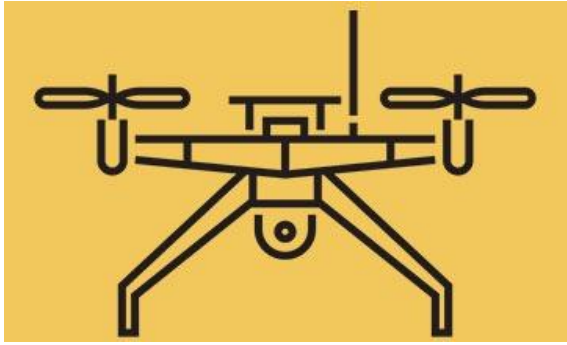
$$\boxed{\hat{\mathbf{x}}_-(k) = \sum_{i=0}^{2n} w^i \boldsymbol{\mathcal{X}}_-^i(k)}$$

Propagate sigma points through the output equation:

$$\boxed{\boldsymbol{\mathcal{Y}}_-^i(k) = \mathbf{h}(\boldsymbol{\mathcal{X}}_-^i(k))}$$

Calculate the corresponding predicted output ("innovation"):

$$\boxed{\hat{\mathbf{y}}_-(k) = \sum_{i=0}^{2n} w^i \boldsymbol{\mathcal{Y}}_-^i(k)}$$



UKF: the prediction step (2/2)

Calculate prior covariance matrix:

Recall that

$$\mathbf{P}_-(k) = \mathrm{E}[(\mathbf{x}(k) - \hat{\mathbf{x}}_-(k))(\mathbf{x}(k) - \hat{\mathbf{x}}_-(k))^T]$$
$$= \mathrm{E}[(\mathbf{f}(\mathbf{x}(k-1)) + \mathbf{w}(k-1) - \hat{\mathbf{x}}_-(k))(\mathbf{f}(\mathbf{x}(k-1)) + \mathbf{w}(k-1) - \hat{\mathbf{x}}_-(k))^T]$$

which gives

$$\mathbf{P}_-(k) = \mathrm{E}[(\mathbf{f}(\mathbf{x}(k-1)) - \hat{\mathbf{x}}_-(k))(\mathbf{f}(\mathbf{x}(k-1)) - \hat{\mathbf{x}}_-(k))^T] + \underbrace{\mathrm{E}[\mathbf{w}(k-1)\mathbf{w}^T(k-1)]}_{\mathbf{Q}(k-1)} \qquad (1)$$

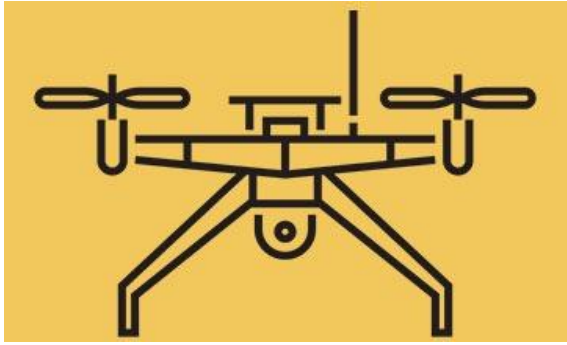
(after cross-term elimination)

BUT f(x(k-1)) is not known...

→ equation (1) is replaced with its UT version:

$$\boxed{\mathbf{P}_-(k) = \sum_{i=0}^{2n} w^i(\boldsymbol{\mathcal{X}}_-^i(k) - \hat{\mathbf{x}}_-(k))(\boldsymbol{\mathcal{X}}_-^i(k) - \hat{\mathbf{x}}_-(k))^T + \mathbf{Q}(k-1)}$$

UKF: the Kalman gain (1/2)



Recall that we had

$$\mathbf{K} = \underline{\mathbf{P}_-(k)\mathbf{C}^T}\,\underline{\left[\mathbf{C}\mathbf{P}_-(k)\mathbf{C}^T + \mathbf{R}\right]}^{-1}$$

but for nonlinear systems, we had $\mathbf{C} = \left[\left.\frac{\partial \mathbf{h}}{\partial \mathbf{x}}\right|_{\mathbf{x}=\hat{\mathbf{x}}}\right]$

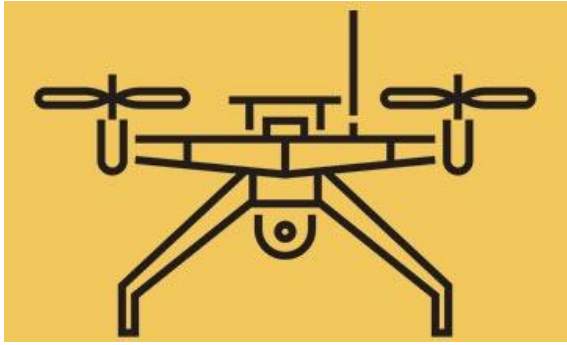
which we don't want to use...

Let us come back to the linear case and look at each term separately:

$$\begin{aligned}
\underline{\mathbf{P}_-(k)\mathbf{C}^T} &= \mathrm{E}[(\mathbf{x}(k) - \hat{\mathbf{x}}_-(k))(\mathbf{x}(k) - \hat{\mathbf{x}}_-(k))^T]\mathbf{C}^T \\
&= \mathrm{E}[(\mathbf{x}(k) - \hat{\mathbf{x}}_-(k))(\mathbf{x}(k) - \hat{\mathbf{x}}_-(k))^T]\mathbf{C}^T + \underbrace{\mathrm{E}[\tilde{\mathbf{x}}_-(k)\mathbf{v}^T(k)]}_{=0 \text{ (uncorrelated)}} \\
&= \mathrm{E}[(\mathbf{x}(k) - \hat{\mathbf{x}}_-(k))(\mathbf{C}\mathbf{x}(k) + \mathbf{v}(k) - \mathbf{C}\hat{\mathbf{x}}_-(k))^T] \\
&= \mathrm{E}[(\mathbf{x}(k) - \hat{\mathbf{x}}_-(k))(\mathbf{y}(k) - \hat{\mathbf{y}}_-(k))^T] \\
&=: \mathbf{P}_{\mathbf{xy}}(k)
\end{aligned}$$

cross-covariance between x and y

UKF: the Kalman gain (2/2)



"Innovation term":

$$\begin{aligned}\underline{\mathbf{C}\mathbf{P}_-(k)\mathbf{C}^T + \mathbf{R}} &= \mathbf{C}\,\mathrm{E}[(\mathbf{x}(k) - \hat{\mathbf{x}}_-(k))(\mathbf{x}(k) - \hat{\mathbf{x}}_-(k))^T]\mathbf{C}^T + \mathrm{E}[\mathbf{v}(k)\mathbf{v}^T(k)] \\ &= \mathrm{E}[(\mathbf{C}\mathbf{x}(k) + \mathbf{v}(k) - \mathbf{C}\hat{\mathbf{x}}_-(k))(\mathbf{C}\mathbf{x}(k) + \mathbf{v}(k) - \mathbf{C}\hat{\mathbf{x}}_-(k))^T] \\ &= \mathrm{E}[(\mathbf{y}(k) - \hat{\mathbf{y}}_-(k))(\mathbf{y}(k) - \hat{\mathbf{y}}_-(k))^T] \\ &=: \mathbf{P}_{\mathbf{y}}(k)\end{aligned}$$

covariance on output = "innovation covariance"

Hence an alternative expression for K is

$$\boxed{\mathbf{K} = \mathbf{P}_{\mathbf{xy}}(k)\mathbf{P}_{\mathbf{y}}^{-1}(k)}$$

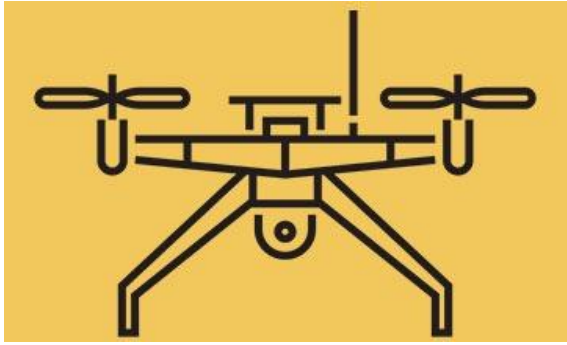
How to obtain $\mathbf{P}_{\mathbf{xy}}(k)$ and $\mathbf{P}_{\mathbf{y}}(k)$? Using sigma points!

(similar approach as starting from (1) in s.11)

$$\boxed{\mathbf{P}_{\mathbf{xy}}(k) = \sum_{i=0}^{2n} w^i(\boldsymbol{\mathcal{X}}_-^i(k) - \hat{\mathbf{x}}_-(k))(\boldsymbol{\mathcal{Y}}_-^i(k) - \hat{\mathbf{y}}_-(k))^T}$$

$$\boxed{\mathbf{P}_{\mathbf{y}}(k) = \sum_{i=0}^{2n} w^i(\boldsymbol{\mathcal{Y}}_-^i(k) - \hat{\mathbf{y}}_-(k))(\boldsymbol{\mathcal{Y}}_-^i(k) - \hat{\mathbf{y}}_-(k))^T + \mathbf{R}(k)}$$

UKF: Correction/update step



The state estimate is the same as the one from a conventional KF:

$$\boxed{\hat{\mathbf{x}}(k) = \hat{\mathbf{x}}_{-}(k) + \mathbf{K}\left[\mathbf{y}(k) - \hat{\mathbf{y}}_{-}(k)\right]}$$

In the KF, we also have

$$\mathbf{P}(k) = \left[\mathbf{I} - \mathbf{K}\mathbf{C}\right]\mathbf{P}_{-}(k)$$

but we do not want to use C

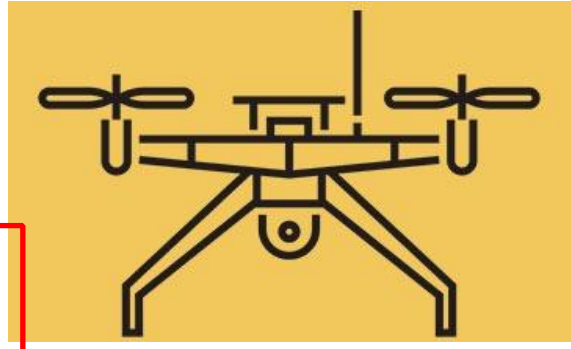
Rewrite P(k) as

$$\begin{aligned}\mathbf{P}(k) &= \mathbf{P}_{-}(k) - \mathbf{K}\underline{\mathbf{C}\mathbf{P}_{-}(k)} \\ &= \mathbf{P}_{-}(k) - \mathbf{K}\mathbf{P}_{\mathbf{xy}}^{T}(k) \\ &= \mathbf{P}_{-}(k) - \mathbf{K}\mathbf{P}_{\mathbf{y}}(k)\underbrace{\mathbf{P}_{\mathbf{y}}^{-1}(k)\mathbf{P}_{\mathbf{xy}}^{T}(k)}_{\mathbf{K}^{T}}\end{aligned}$$

So that we finally have

$$\boxed{\mathbf{P}(k) = \mathbf{P}_{-}(k) - \mathbf{K}\mathbf{P}_{\mathbf{y}}(k)\mathbf{K}^{T}}$$

UKF: the big picture



Prediction step

$$\boldsymbol{\mathcal{X}}^0(k-1) = \hat{\mathbf{x}}(k-1)$$

$$\boldsymbol{\mathcal{X}}^i(k-1) = \hat{\mathbf{x}}(k-1) \pm \sqrt{(n+\kappa)}\left[\sqrt{\mathbf{P}(k-1)}\right]_i$$

$$\boldsymbol{\mathcal{X}}^i_-(k) = \mathbf{f}(\boldsymbol{\mathcal{X}}^i(k-1)),\ \boldsymbol{\mathcal{Y}}^i_-(k) = \mathbf{h}(\boldsymbol{\mathcal{X}}^i_-(k))$$

$$\hat{\mathbf{x}}_-(k) = \sum_{i=0}^{2n} w^i \boldsymbol{\mathcal{X}}^i_-(k),\ \hat{\mathbf{y}}_-(k) = \sum_{i=0}^{2n} w^i \boldsymbol{\mathcal{Y}}^i_-(k)$$

$$\mathbf{P}_-(k) = \sum_{i=0}^{2n} w^i (\boldsymbol{\mathcal{X}}^i_-(k) - \hat{\mathbf{x}}_-(k))(\boldsymbol{\mathcal{X}}^i_-(k) - \hat{\mathbf{x}}_-(k))^T + \mathbf{Q}(k-1)$$

Kalman gain

$$\mathbf{P}_{\mathbf{xy}}(k) = \sum_{i=0}^{2n} w^i (\boldsymbol{\mathcal{X}}^i_-(k) - \hat{\mathbf{x}}_-(k))(\boldsymbol{\mathcal{Y}}^i_-(k) - \hat{\mathbf{y}}_-(k))^T$$

$$\mathbf{P}_{\mathbf{y}}(k) = \sum_{i=0}^{2n} w^i (\boldsymbol{\mathcal{Y}}^i_-(k) - \hat{\mathbf{y}}_-(k))(\boldsymbol{\mathcal{Y}}^i_-(k) - \hat{\mathbf{y}}_-(k))^T + \mathbf{R}(k)$$

$$\mathbf{K} = \mathbf{P}_{\mathbf{xy}}(k)\mathbf{P}_{\mathbf{y}}^{-1}(k)$$

Correction/update step

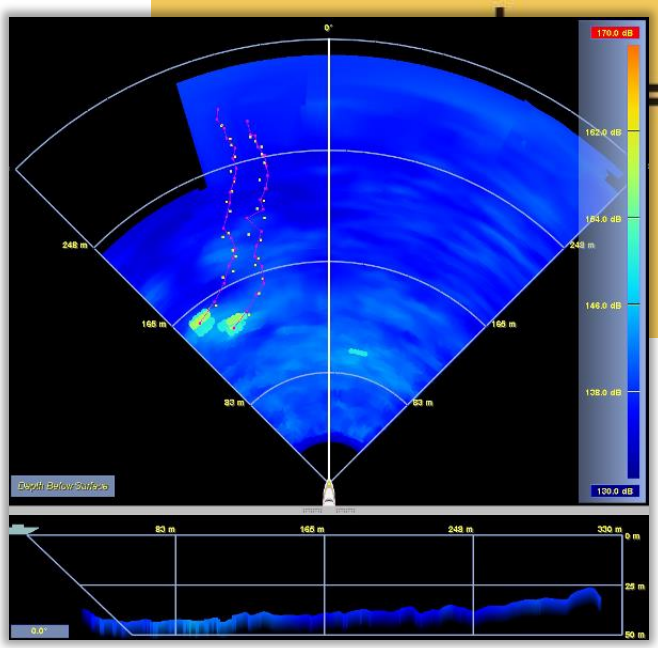
$$\hat{\mathbf{x}}(k) = \hat{\mathbf{x}}_-(k) + \mathbf{K}\left[\mathbf{y}(k) - \hat{\mathbf{y}}_-(k)\right]$$

$$\mathbf{P}(k) = \mathbf{P}_-(k) - \mathbf{K}\mathbf{P}_{\mathbf{y}}(k)\mathbf{K}^T$$

Let's take a break

"That's the main result. Now comes the fun part... Let's obscure it with lots of symbols, new definitions, and unnecessary jargon so that it will take 'em until mesolithic age to figure it out!"

Back to our sonar example



- Q1) In Matlab, re-implement the sonar range-bearing scenario (s. 6) by defining range and bearing signals with the same mean and covariance matrix.
- Q2) Transform these two signals into the cartesian space and visualise the distribution both in 2D and 3D (use 'hist3' for the latter).
- Q3) Implement sigma points and propagate them into your nonlinear transformation between polar and cartesian coordinates. Add these sigma points with their weight in your 3D plot.
- Q4) Draw both 'Unscented' mean and covariance matrix (you can use the function uploaded on Itslearning called 'plotEllipse.m')

Underwater navigation

- Q5) Consider an AUV cruising at constant velocity in a horizontal plane (ie at constant depth), and modelled by the differential equation $\ddot{\mathbf{x}} = 0$ Discretize this model and implement it in a subsystem in Simulink.
- Q6) This AUV evolves between 2 acoustic beacons of positions (0,0) and (50,0), respectively. Implement the fact that the AUV measures range distances with them, ie we have $\mathbf{y} = \begin{bmatrix} \sqrt{(\mathbf{x}-\mathbf{b}_1)^T(\mathbf{x}-\mathbf{b}_1)} \\ \sqrt{(\mathbf{x}-\mathbf{b}_2)^T(\mathbf{x}-\mathbf{b}_2)} \end{bmatrix}$
- Q7) Implement an Unscented Kalman Filter to estimate the horizontal position and velocity of this vehicle.
- Q8) Modify your UKF so that it can use additional onboard velocity measurements
- Q9) Modify your UKF to the 3D case.